

Single-Trace Message Recovery in HQC via RS Post-Decoding and FO Re-Encryption

Jaeho Jeon¹, Donghyen Kim¹, Suseong Lee¹ and Young-Sik Kim¹

Department of Electrical Engineering and Computer Science, DGIST, Daegu, Korea,
dgwogh@dgist.ac.kr, dhkim200426@dgist.ac.kr, mercury@dgist.ac.kr, ysk@dgist.ac.kr

Abstract. HQC is a code-based key-encapsulation mechanism standardized by NIST, whose decapsulation follows a Fujisaki–Okamoto (FO) transform and re-executes encryption-side encoding during deterministic re-encryption. In this work, we show that this design, combined with the concatenated RS/RM encoding structure, enables efficient single-trace message recovery from side-channel leakage.

We identify a strong and structured leakage source in the Reed–Muller (RM) encoding routine, which is preserved across the NIST-submitted implementations, the HQC team’s official codebase, and the PQClean implementations. Building on this observation, we move beyond direct recovery of the 16 systematic message bytes and instead target the full 46-byte Reed–Solomon (RS) codeword. By exploiting RS decoding as a post-correction mechanism, we show that approximate recovery of RM-input symbols is sufficient to reconstruct the original message with high probability.

Our results show that RS post-correction fundamentally changes the attack regime. Without it, direct message recovery remains highly sensitive to residual byte errors and degrades substantially on noisier measurement platforms. With RS post-correction, however, the attack becomes robust across devices: on a ChipWhisperer CW308 UFO board with an STM32F303 target, we recover the full 128-bit encapsulation message from a *single* decapsulation trace with up to 98.9% success using only 20 profiling traces, while on the noisier STM32F415 platform we achieve 99.5% successful RS-corrected recovery with only 60 profiling traces.

These results show that the outer RS code, originally introduced to reduce the decryption failure rate, can paradoxically facilitate side-channel message recovery by tolerating residual symbol errors. More broadly, our results highlight how reliability-oriented code design and FO-style re-encryption can interact to create powerful side-channel attack surfaces in code-based KEMs.

Keywords: HQC · Hamming Quasi-Cyclic · PQC · Post-Quantum Cryptography · KEM · Embedded systems · side-channel attacks

1 Introduction

The advancement of quantum computing presents a significant threat to classical public-key cryptographic systems such as RSA [RSA78] and ECC [Mil86]. In particular, Shor’s algorithm [Sho94] has demonstrated the ability to efficiently break these schemes in polynomial time on a quantum computer. In response, the US National Institute of Standards and Technology (NIST) initiated the Post-Quantum Cryptography (PQC) [BL17] standardization process [BL17, CCJ⁺16]. As a result, in 2022, several PQC schemes, such as the lattice-based key encapsulation mechanism (KEM) CRYSTALS-Kyber [BDK⁺18], the lattice-based signature schemes CRYSTALS-Dilithium [DKL⁺18] and Falcon [FHK⁺18], and the stateless hash-based signature scheme SPHINCS+ [BHK⁺19], were selected as new NIST PQC standards. Recognizing that diversified assumptions about computational hardness are essential for robust cryptographic security, the standardization process emphasized

the inclusion of schemes based on diverse mathematical foundations [AAC⁺22]. Accordingly, the fourth-round KEM finalists featured code-based candidates such as BIKE [ABB⁺22], Classic McEliece [BCL⁺17], and Hamming Quasi-Cyclic (HQC) [MAB⁺18], with HQC officially adopted as a new standard in 2025 [ABC⁺25].

Beyond theoretical robustness, NIST has underscored the importance of practical security in real deployments, including embedded and resource-constrained platforms [AAC⁺22]. In such settings, side-channel leakage through power, electromagnetic emanations, or timing variations can undermine the security of otherwise sound cryptographic designs. Consequently, implementation security—and in particular resistance against side-channel attacks—is a critical requirement for post-quantum cryptography intended for widespread adoption.

1.1 Message and Ephemeral Secret Recovery in PQC

While physical attacks on PQC have traditionally focused on recovering *long-term secret keys*, prior work shows that *ephemeral secrets*—most notably the encapsulation/encryption message—can be equally damaging when implementations leak during encoding or decoding. In lattice-based KEMs, such leakage enables *single-trace message recovery* (e.g., Kyber and Saber), and in KEMs built via the Fujisaki–Okamoto transform (FO transform) [FO99] the recovered message often suffices to recompute the shared session key from public transcript data [SKL⁺20]. Ravi *et al.* further systematized this threat across NIST PQC candidates, showing that even *partial* message recovery can substantially simplify follow-up attacks, including chosen-ciphertext-assisted recovery of long-term secrets [RBRC21].

Message/plaintext recovery is also practical in code-based cryptography. Lahr *et al.* demonstrate partial plaintext recovery for Classic McEliece via side-channel-assisted information-set decoding [LNPS20], and related profiled attacks confirm that message-related leakage can be exploited in syndrome-decoding settings [CDCG22]. For HQC, Goy *et al.* achieve *single-trace* shared-key recovery using SASCA methods [VCGS14, GMGL24] by exploiting leakage from Reed–Solomon finite-field multiplication (e.g., `gf_mul`) and performing probabilistic inference. Moreover, message recovery can persist even under countermeasures: Ngo *et al.* recover the encapsulation message from a *first-order masked*, *IND-CCA-secure* Saber implementation using deep-learning-based analysis, and leverage it toward long-term key recovery [NDGJ21]; similar encoding/decoding-dependent leakage has also been analyzed for FrodoKEM [Ber25].

These results are particularly relevant in the KEM setting because FO-style transforms re-execute encryption-side computations during decapsulation via deterministic re-encryption. Consequently, message-dependent encoding/decoding routines may leak even when the adversary can only observe decapsulation [FO99, RBRC21, Ber25]. This observation motivates revisiting HQC with a focus on message-dependent leakage in its encoding pipeline.

1.2 Oracle-Based Side-Channel Attacks on HQC

During the NIST PQC process, HQC has become a prominent target for oracle-based side-channel cryptanalysis. A series of works progressively refined how to construct and exploit plaintext-checking and decryption oracles, leading to increasingly efficient full secret-key recovery attacks.

Early practical attacks exploited cache-timing leakage to instantiate a plaintext-checking (PC) oracle that distinguishes valid from invalid decryptions, showing that even ostensibly constant-time implementations may leak enough information to bypass CCA security in a chosen-ciphertext setting [HSC⁺23]. Subsequent work demonstrated that the oracle abstraction is largely agnostic to the leakage source: the divide-and-surrender attack

leveraged subtle instruction-level timing variations in arithmetic operations to build higher-accuracy PC oracles and to reduce the number of required oracle queries well beyond earlier cache-based approaches [SGG24].

More recently, the emphasis shifted from *finding* new leakage sources to maximizing the *information extracted per oracle call*. OT-PCA introduced an offline-template approach that derives soft information from Reed–Muller decoding outcomes, enabling full key recovery with orders of magnitude fewer oracle calls even under imperfect oracle accuracy [DG25b]. In parallel, multi-value plaintext-checking and full-decryption oracles further generalized the oracle model so that each query can convey richer information about multiple secret positions simultaneously [DG25a].

Overall, oracle-based attacks on HQC have developed into a theoretically mature and highly effective framework: the oracle models have been systematically refined, and the resulting key-recovery methods achieve markedly improved efficiency.

1.3 Motivation and Key Observation

The preceding discussion highlights two complementary trends in the side-channel analysis of post-quantum cryptography. On the one hand, message and ephemeral-secret recovery attacks show that leakage in encoding and decoding can directly compromise session keys in KEM constructions. On the other hand, recent attacks on HQC show that the interaction between implementation leakage and coding structure can be exploited in multiple ways, ranging from message recovery to long-term secret-key recovery.

Message recovery in HQC is already a concrete threat. Message recovery in HQC has recently been demonstrated in several forms. Goy *et al.* achieve single-trace shared-key recovery using SASCA by exploiting leakage in the Reed–Solomon finite-field multiplication routine (e.g., `gf_mul`) and performing belief-propagation-based inference [GMGL24]. Earlier, Goy *et al.* also presented a horizontal correlation attack against the RS decoding procedure of HQC, showing that repeated manipulations of RS-codeword bytes within a single trace can be exploited and that the public RS decoder itself can be used to correct side-channel inference errors [GLG22]. Taken together, these results establish that message-dependent leakage in HQC is not merely a theoretical concern, but a practical attack surface supported by the structure of the concatenated code.

From RM-encoding leakage to RS-aware recovery. In this work, we identify the Reed–Muller (RM) encoding routine (RM(1, 7) in HQC) as a distinct leakage source [MAB⁺18, MS77]. In contrast to previous leakage targets, the RM encoder exhibits highly structured, bit-level data dependence due to bit-to-word mask expansions in the optimized implementation. This yields strong and temporally separable features that make the RM-input symbols directly recoverable from side-channel traces.

Our key observation is that the outer Reed–Solomon (RS) code in HQC does more than improve reliability against transmission noise: from an attacker’s perspective, it also acts as a post-correction mechanism. In a direct message-byte recovery setting, residual byte errors quickly destroy the full-message success rate, even when per-byte accuracy is moderately high. By contrast, if the attacker recovers a noisy estimate of the full 46-byte RS codeword, the outer RS decoder can absorb up to $\delta = 15$ symbol errors and still reconstruct the original 128-bit message exactly. Thus, the attack objective shifts from perfect recovery of the 16 systematic message bytes to approximate recovery of the full RS codeword.

A reliability–security paradox under FO re-encryption. This creates a paradoxical effect. The outer RS code is introduced in HQC to reduce the decryption failure rate

and thereby strengthen reliability. However, under side-channel observation, the same error-correction capability increases the attacker’s tolerance to residual inference errors and substantially lowers the effort required for successful message recovery. Moreover, because HQC decapsulation re-executes encryption-side encoding through FO-style deterministic re-encryption, this leakage becomes observable even when the adversary can measure only decapsulation traces. In this sense, reliability-oriented code design and FO-style re-encryption interact to create a particularly favorable attack surface for single-trace message recovery.

1.4 Our Contributions

Our findings apply to the HQC team’s latest upstream implementation, the NIST fourth-round optimized implementation, and the corresponding PQClean implementations on Cortex-M4-class microcontrollers, and persist under common embedded build settings (both `-O3` and `-Os`). We show that single-trace message recovery in HQC is enabled not only by a strong leakage source in RM encoding, but also by the surrounding protocol and code structure: FO re-encryption makes the leakage observable during decapsulation, and the outer RS code turns approximate symbol recovery into highly reliable message recovery. Our main contributions are as follows.

- **A structured and cross-implementation leakage source in HQC RM encoding.** We identify strong, structured leakage in the RM(1, 7) encoder of HQC, caused by message-bit expansion into full-word masks in optimized Cortex-M4 implementations. This produces temporally separable, message-dependent leakage points that are preserved across the HQC team’s upstream codebase, the NIST-submitted implementations, and the corresponding PQClean implementations.
- **RS-aware single-trace message recovery via full-codeword reconstruction.** Building on this leakage source, we move beyond direct recovery of the 16 systematic message bytes and instead target the full 46-byte Reed–Solomon codeword processed by RM encoding. We then exploit outer RS decoding as a post-correction mechanism, so that approximate recovery of RM-input symbols suffices to reconstruct the original 128-bit message. This turns message recovery into an RS-aware attack pipeline rather than a pure byte-wise classification problem.
- **RS post-correction dramatically strengthens recovery under noisy conditions.** Without RS post-correction, direct message recovery is highly sensitive to residual byte errors: on STM32F303, raw recovery reaches up to **96.9%** full-message success with **3,200** profiling traces, whereas on the noisier STM32F415 platform it remains limited to **17.2%** even with **2,000** profiling traces. In contrast, once RS post-correction is applied to the recovered 46-byte codeword, the attack becomes robust across platforms: we achieve up to **98.9%** full-message recovery on STM32F303 with only **20** profiling traces, and **99.5%** successful RS-based recovery on STM32F415 with only **60** profiling traces. This shows that the outer RS code, originally introduced to reduce the decryption failure rate, paradoxically facilitates side-channel message recovery by tolerating residual symbol errors in the attacker’s inference.
- **A design-level explanation of why HQC becomes vulnerable in decapsulation.** We show that the practical attack surface arises from the interaction of three design elements: RM encoding provides a strong implementation-level leakage source, FO-style deterministic re-encryption re-executes this leakage-prone encoding during decapsulation, and outer RS decoding amplifies the attack by correcting residual symbol errors. This yields a reliability–security paradox in which a component

introduced to improve correctness also increases recoverability under side-channel observation.

2 Preliminaries

2.1 Overview of HQC

We summarize the HQC-PKE algorithms at a high level, omitting implementation details related to XOF initialization, seed expansion, and the implementation-specific compression/truncation of ciphertext components. We focus instead on the algebraic structure relevant to oracle construction and message leakage.

Key generation. HQC-PKE key generation samples two sparse secret vectors x, y and a dense public vector h , and computes $s = x + h \cdot y$. The public key is $\text{pk} = (h, s)$ and the secret key contains $\text{sk} = (x, y)$ (and associated seeds in the concrete implementation).

Algorithm 1 HQC-PKE.Keygen (simplified)

Require: Parameters $(\mathcal{R}, \mathcal{R}_\omega)$

Ensure: Public key $\text{pk} = (h, s)$ and secret key $\text{sk} = (x, y)$

- 1: Sample $x \leftarrow \text{SampleFixedWeightVect}(\mathcal{R}_\omega)$ $\triangleright x$ sparse
 - 2: Sample $y \leftarrow \text{SampleFixedWeightVect}(\mathcal{R}_\omega)$ $\triangleright y$ sparse
 - 3: Sample $h \leftarrow \text{SampleVect}(\mathcal{R})$ $\triangleright h$ dense
 - 4: $s \leftarrow x + h \cdot y$
 - 5: **return** $\text{pk} = (h, s), \text{sk} = (x, y)$
-

Encryption. To encrypt a message m under $\text{pk} = (h, s)$, the algorithm samples sparse ephemeral values (r_1, r_2, e) and outputs a ciphertext $c_{\text{PKE}} = (u, v)$ where

$$u = r_1 + h \cdot r_2, \quad v = \text{C.Encode}(m) + s \cdot r_2 + e.$$

Here, $\text{C.Encode}(\cdot)$ denotes HQC's concatenated RS/RM encoding, whose implementation is analyzed in Section 3.

Algorithm 2 HQC-PKE.Encrypt (simplified)

Require: Public key $\text{pk} = (h, s)$, message m

Ensure: Ciphertext $c_{\text{PKE}} = (u, v)$

- 1: Sample $r_1 \leftarrow \text{SampleFixedWeightVect}(\mathcal{R}_\omega)$ $\triangleright r_1$ sparse
 - 2: Sample $r_2 \leftarrow \text{SampleFixedWeightVect}(\mathcal{R}_\omega)$ $\triangleright r_2$ sparse
 - 3: Sample $e \leftarrow \text{SampleFixedWeightVect}(\mathcal{R}_\omega)$ $\triangleright e$ sparse
 - 4: $u \leftarrow r_1 + h \cdot r_2$
 - 5: $v \leftarrow \text{C.Encode}(m) + s \cdot r_2 + e$
 - 6: **return** $c_{\text{PKE}} = (u, v)$
-

Decryption. Given $c_{\text{PKE}} = (u, v)$ and secret y , decryption computes

$$m = \text{C.Decode}(v - u \cdot y),$$

where $\text{C.Decode}(\cdot)$ denotes HQC's concatenated RS/RM decoding. This affine structure will also be relevant for understanding how decoding acts on noisy symbol estimates and why message-dependent behavior in the encoding and decoding pipeline can have security consequences.

Algorithm 3 HQC-PKE.Decrypt (simplified)**Require:** Secret key sk containing y , ciphertext $c_{\text{PKE}} = (u, v)$ **Ensure:** Message m

- 1: $m \leftarrow \text{C.Decode}(v - u \cdot y)$
- 2: **return** m

KEM decapsulation and FO re-encryption. HQC-KEM applies an FO-style transform on top of HQC-PKE. Decapsulation first computes a candidate message $m' \leftarrow \text{HQC-PKE.Decrypt}(\text{sk}_{\text{PKE}}, c_{\text{PKE}})$, then deterministically derives both the candidate shared key and the re-encryption randomness as $(K', \theta') \leftarrow G(H(\text{pk}_{\text{KEM}}) \parallel m' \parallel \text{salt})$, and re-encrypts m' as $c'_{\text{PKE}} \leftarrow \text{HQC-PKE.Encrypt}(\text{pk}_{\text{KEM}}, m', \theta')$ to validate the ciphertext. If $c'_{\text{PKE}} = c_{\text{PKE}}$ (and $m' \neq \perp$), the output is $K = K'$; otherwise the implementation returns a rejection key $\bar{K} \leftarrow J(H(\text{pk}_{\text{KEM}}) \parallel \sigma \parallel c_{\text{KEM}})$. Crucially, the deterministic re-encryption re-executes $\text{C.Encode}(m')$, which is the leakage point studied in this work.

Algorithm 4 HQC-KEM.Decaps (FO transform, simplified)**Require:** Secret key sk_{KEM} (contains sk_{PKE} , pk_{KEM} , and σ), ciphertext c_{KEM} (contains c_{PKE} and salt)**Ensure:** Shared key K

- 1: $m' \leftarrow \text{HQC-PKE.Decrypt}(\text{sk}_{\text{PKE}}, c_{\text{PKE}})$
- 2: $(K', \theta') \leftarrow G(H(\text{pk}_{\text{KEM}}) \parallel m' \parallel \text{salt})$
- 3: $c'_{\text{PKE}} \leftarrow \text{HQC-PKE.Encrypt}(\text{pk}_{\text{KEM}}, m', \theta')$ ▷ re-encryption
- 4: $\bar{K} \leftarrow J(H(\text{pk}_{\text{KEM}}) \parallel \sigma \parallel c_{\text{KEM}})$ ▷ rejection key
- 5: **if** $m' \neq \perp$ **and** $c'_{\text{PKE}} = c_{\text{PKE}}$ **then**
- 6: $K \leftarrow K'$
- 7: **else**
- 8: $K \leftarrow \bar{K}$
- 9: **end if**
- 10: **return** K

The highlighted re-encryption step re-executes $\text{C.Encode}(m')$ during decapsulation, providing the observation point for both our message-recovery attack and the oracle instantiation discussed later.

2.2 Concatenated RS/RM Encoding and Decoding in HQC (HQC-128)

As introduced in Section 2.1, HQC encodes the 128-bit encapsulation/encryption message $\mathbf{m}$ using a concatenated error-correcting code $\text{C.Encode}(\cdot)$: an outer Reed–Solomon (RS) code over $\mathbb{F}_{256}$ followed by an inner (binary) Reed–Muller (RM) code (Fig. 1) [MAB⁺18]. Intuitively, the RS layer protects the message at the *symbol* (byte) level, while the RM layer protects each RS symbol at the *bit* level before transmission.

Encoding: RS then RM (with duplication). For HQC-128, the RS encoder takes the 128-bit message m (i.e., $k = 16$ bytes) and produces an RS codeword of length $n_1 = 46$ bytes, expanding the payload from $16 \times 8 = 128$ bits to $46 \times 8 = 368$ bits. Moreover, HQC uses a *systematic* RS encoder, meaning the message bytes appear *verbatim* in fixed positions of the RS codeword:

$$\text{cdw}[30..45] = m[0..15], \quad \text{cdw}[0..29] \text{ are parity symbols.}$$

In other words, the red-boxed region in Fig. 1 corresponds exactly to the systematic part of the RS codeword that directly contains m .

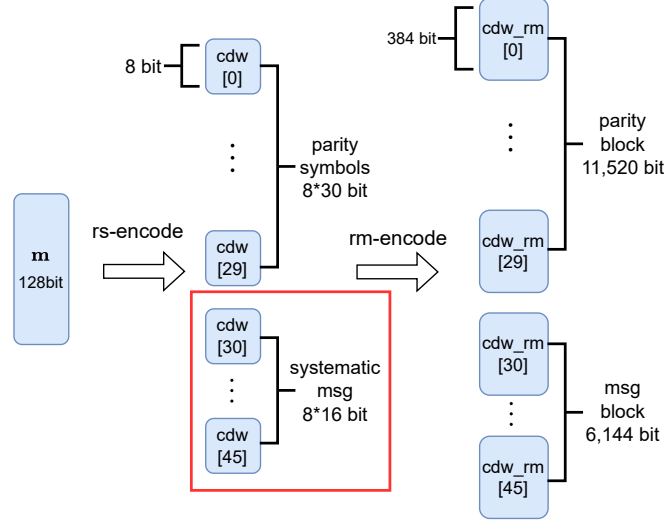


Figure 1: Concatenated RS/RM encoding in HQC (HQC-128 parameters). A 128-bit message m is first encoded by a systematic Reed-Solomon code to form a 46-byte RS codeword $\text{cdw}[0..45]$, where $\text{cdw}[0..29]$ are parity symbols and $\text{cdw}[30..45]$ copy the message bytes verbatim. Each 8-bit RS symbol is then independently Reed-Muller encoded (RM(1, 7)) and duplicated ($d = 3$), producing a 384-bit block per symbol. Concatenating all 46 blocks yields $46 \times 384 = 17,664$ encoded bits, which can be viewed as a parity block (11,520 bits) and a message block (6,144 bits) corresponding to the systematic region.

After RS encoding, HQC applies the inner RM encoder independently to each RS symbol. Concretely, HQC uses the first-order RM code RM(1, 7), which maps one 8-bit input symbol to a 128-bit binary codeword. HQC then applies a duplication factor $d = 3$ (bit repetition), so each RS symbol becomes a $128 \cdot 3 = 384$ -bit block. Thus, encoding the full RS codeword of 46 symbols yields a total of

$$46 \times 384 = 17,664 \text{ bits,}$$

which naturally decomposes into a parity block ($30 \times 384 = 11,520$ bits) and a message block ($16 \times 384 = 6,144$ bits), mirroring the systematic RS partition.

From encoding to encryption noise. In HQC-PKE encryption, the binary codeword $\text{C.Encode}(m)$ is then combined with ciphertext-dependent terms and sparse noise (error injection). As a result, the receiver observes a noisy version of the concatenated codeword, which must be corrected by $\text{C.Decode}(\cdot)$.

Decoding: RM block correction then RS symbol correction. Decoding proceeds in the reverse order. First, RM decoding is applied *block-wise* to each duplicated RM chunk $\text{cdw_rm}[i]$ for $i \in \{0, \dots, 45\}$. HQC uses the first-order Reed-Muller code RM(1, 7) with parameters $[128, 8, 64]$, and then duplicates each bit d times (for HQC-128, $d = 3$), yielding a duplicated RM code of parameters $[384, 8, 192]$ (minimum distance $d_{\min} = 192$). In particular, in the standard bounded-distance view, a length-384 RM block can correct up to $\lfloor (d_{\min} - 1)/2 \rfloor = 95$ bit errors within the block. Operationally, each block is decoded via a Hadamard-transform-based procedure adapted to duplication, outputting an 8-bit estimate of the corresponding RS symbol. Depending on the noise realization, an RM block may decode successfully (yielding a reliable symbol) or fail/produce a noisy estimate.

Second, RS decoding operates on the recovered 46-byte symbol sequence and corrects remaining errors at the *symbol* level. In HQC-128, the outer code is the shortened, systematic Reed–Solomon code RS-S1 with parameters $(n_1, k, d_{\min}) = (46, 16, 31)$. Equivalently, its correcting capacity is $\delta = 15$ (since $n_1 - k = 2\delta$), and shortening does not change this capability. Accordingly, the RS decoder can correct up to $\delta = 15$ erroneous RS symbols across the 46-symbol word, after which the original message m is read from the systematic part of the corrected RS codeword.

Attack perspective: RS decoding as post-correction. From an attacker’s perspective, the outer RS layer can be repurposed as a post-correction mechanism. In particular, once the attacker recovers a noisy estimate of the 46-byte RS codeword, exact recovery of all 16 systematic message bytes is no longer necessary. Instead, it suffices that the inferred 46-byte word contains at most $\delta = 15$ symbol errors, in which case RS decoding reconstructs the original codeword and hence the original 128-bit message exactly. This observation fundamentally changes the attack objective: rather than solving a strict all-bytes-correct message-recovery problem, the attacker only needs approximate full-codeword recovery within the correction radius of the outer RS decoder. This attack-oriented use of RS decoding is consistent with prior observations by Goy *et al.*, who showed that the public RS decoder can be used to correct erroneous byte estimates obtained from side-channel analysis [GLG22].

3 Message Leakage via Reed–Muller Encoding in HQC

This section explains leakage sources arising from the Reed–Muller (RM) encoding routine in HQC and shows how the leakage enables recovery of message-dependent information from a single execution trace.

Our analysis targets the official HQC reference implementation released by the HQC team, specifically the GitLab upstream version tagged v5.0.0 (Aug. 22, 2025) and the subsequent commit d622142a (Aug. 26, 2025), available from their public GitLab repository (<https://gitlab.com/pqc-hqc/hqc/>). [GAMA⁺25, Tea26] Unless stated otherwise, all code-level observations and implementation details reported in this section refer to that codebase.

Our analysis targets the upstream HQC-team reference implementation. We note, however, that its RM-encoding structure is preserved in the NIST-submitted reference and `optimized_implementation` as well as in the corresponding PQClean implementations. Therefore, the same vulnerable behavior (and hence the attack surface) carries over to these codebases.

3.1 Re-encryption and Message Encoding in HQC

HQC decapsulation follows a Fujisaki–Okamoto (FO) style transform: it first decrypts a PKE ciphertext to obtain a candidate message m' , and then deterministically *re-encrypts* m' to verify ciphertext validity (Algorithm 4):

$$m' \leftarrow \text{HQC-PKE.Decrypt}(\text{dk}_{\text{PKE}}, c_{\text{PKE}}), \quad c'_{\text{PKE}} \leftarrow \text{HQC-PKE.Encrypt}(\text{ek}_{\text{PKE}}, m', \theta').$$

This FO re-encryption re-executes the encryption-side message encoding `C.Encode( $m'$ )` (RS then RM with duplication; see Section 2.2), so message-dependent leakage in the encoding pipeline is observable even when the adversary can only measure decapsulation.

As a consequence, a key feature of HQC becomes particularly relevant: the RS layer is implemented in *systematic* form, where the message symbols are copied verbatim into fixed positions of the RS codeword, while the remaining symbols carry parity (Fig. 1).

In particular, for HQC-128 the RS output satisfies $\text{cdw}[30..45] = m[0..15]$, so the subsequent RM encoder processes these message bytes *as part of* its per-symbol inputs when producing the corresponding RM blocks. For our analysis, this creates a simple and fixed correspondence between message bytes and a contiguous region of RS symbols that is immediately processed by RM encoding.

In the HQC reference code, RS encoding is implemented as follows:

Listing 1: Systematic Reed–Solomon encoding in HQC (excerpt).

```

1 void reed_solomon_encode(uint64_t *cdw, const uint64_t *msg) {
2     size_t i, j, k;
3     uint8_t gate_value = 0;
4
5     uint16_t tmp[PARAM_G] = {0};
6     uint16_t PARAM_RS_POLY[] = {RS_POLY_COEFS};
7
8     uint8_t msg_bytes[PARAM_K] = {0};
9     uint8_t cdw_bytes[PARAM_N1] = {0};
10
11     memcpy(msg_bytes, msg, PARAM_K);
12
13     /* RS parity computation: fills cdw_bytes[0 .. PARAM_N1-PARAM_K-1] (
14        omitted) */
15     memcpy(cdw_bytes + PARAM_N1 - PARAM_K, msg_bytes, PARAM_K); /* (1) */
16     memcpy(cdw, cdw_bytes, PARAM_N1);
17 }

```

Inspecting this routine confirms the intended systematic structure: the encoder first computes the parity prefix of the RS codeword (omitted above), where the core arithmetic is driven by finite-field multiplications (`gf_mul`) and subsequent shift/XOR updates of `cdw_bytes`. It then appends the information part by directly copying `msg_bytes` into the last `PARAM_K` symbols via the final copy operation (1), i.e., the message bytes are preserved verbatim in the RS codeword.

Concretely, for the analyzed parameter set, the RS codeword has length `PARAM_N1` = 46 bytes, while the RS message has length `PARAM_K` = 16 bytes (i.e., 128 bits). Due to (1), after RS encoding, the message bytes appear verbatim in the systematic part of the codeword:

$$\text{cdw_bytes}[30..45] = \text{msg_bytes}[0..15],$$

whereas the remaining prefix

$$\text{cdw_bytes}[0..29]$$

contains the redundancy (parity) symbols computed from `msg_bytes`.

This systematic placement will be useful in the following analysis because it makes the message-dependent portion of the RS codeword explicit and byte-aligned before RM encoding. In the next subsection, we show that the subsequent RM encoding routine introduces a strong leakage point that enables direct recovery of the message bytes.

After RS encoding, HQC applies the inner RM encoder independently to each RS symbol. Concretely, the array `cdw_bytes[0..45]` is processed byte-by-byte: for each $i \in \{0, \dots, 45\}$, the encoder takes the 8-bit input `cdw_bytes[i]` and produces a 128-bit RM codeword, so the RM encoding routine is executed a total of 46 times per message encoding.

HQC uses the first-order Reed–Muller code $\text{RM}(1, 7)$ with parameters $[128, 8, 64]$, which can be viewed as a linear mapping from an 8-bit vector to a 128-bit codeword. Equivalently, for an input byte interpreted as a length-8 binary vector, encoding corresponds to multiplying by a fixed generator matrix $G \in \mathbb{F}_2^{8 \times 128}$, i.e., $\mathbf{c} = \mathbf{m}G$ over $\mathbb{F}_2$.

In the HQC implementation, the generator rows of G are realized via compact 32-bit masks that match the standard $\text{RM}(1, 7)$ pattern. The masks are arranged as follows:

```
* 0  aaaaaaaaa aaaaaaaaa aaaaaaaaa aaaaaaaaa
* 1  cccccccc cccccccc cccccccc cccccccc
* 2  f0f0f0f0 f0f0f0f0 f0f0f0f0 f0f0f0f0
* 3  ff00ff00 ff00ff00 ff00ff00 ff00ff00
* 4  ffff0000 ffff0000 ffff0000 ffff0000
* 5  ffffffff 00000000 ffffffff 00000000
* 6  ffffffff ffffffff 00000000 00000000
* 7  ffffffff ffffffff ffffffff ffffffff
```

Rows 0–4 exhibit a 32-bit periodic structure over the 128-bit codeword, while rows 5–6 have 64-bit periodicity, and row 7 is the all-ones mask. This regular structure enables an efficient Cortex-M4 implementation that avoids explicit bit-matrix multiplication over all 128 output positions.

Rather than computing the $\mathbb{F}_2$ -linear combination $\mathbf{m}G$ bit-by-bit, the Cortex-M4 optimized code path uses word-level masking and XORs. The key idea is to expand each message bit into a full-word mask (all-zero or all-one) and then conditionally XOR fixed constants that correspond to generator-row patterns. The resulting routine is shown below.

The last three bits are handled slightly differently: bit 7 initializes an initial pattern, while bits 5 and 6 toggle the full 32-bit word (without an AND mask) to realize longer-period (64/128-bit) patterns across the four 32-bit limbs.

For implementation convenience, HQC expands each input bit into an all-zero or all-one 32-bit mask via $\text{BITOMASK}(x) = -((x) \& 1)$. Concretely, if the selected message bit is 0, BITOMASK evaluates to $0x00000000$; if the bit is 1, it evaluates to $0xFFFFFFFF$. The encoder then combines these masks with fixed constants such as $0xaaaaaaaa$ and $0xcccccccc$ using bitwise AND and XOR. As we show next, this “bit-to-word” expansion leads to a highly regular instruction sequence whose intermediate register values are strongly data dependent.

Listing 2: Reed–Muller encoding routine in HQC (excerpt).

```

1  #define BITOMASK(x) ((int32_t)(-((x) & 1)))
2
3  typedef union {
4      uint8_t u8[16];    /**< Byte-wise access (16 bytes) */
5      uint32_t u32[4];   /**< Word-wise access (4 32-bit words) */
6  } rm_codeword_t;
7
8  void encode(rm_codeword_t *word, int32_t message) {
9      int32_t first_word;
10
11     first_word = BITOMASK(message >> 7);
12     first_word ^= BITOMASK(message >> 0) & 0xaaaaaaaa;
13     first_word ^= BITOMASK(message >> 1) & 0xcccccccc;
14     first_word ^= BITOMASK(message >> 2) & 0xf0f0f0f0;
15     first_word ^= BITOMASK(message >> 3) & 0xff00ff00;
16     first_word ^= BITOMASK(message >> 4) & 0xffff0000;
17     word->u32[0] = first_word;
18
19     first_word ^= BITOMASK(message >> 5);
20     word->u32[1] = first_word;
21     first_word ^= BITOMASK(message >> 6);
22     word->u32[3] = first_word;
23     first_word ^= BITOMASK(message >> 5);
24     word->u32[2] = first_word;
25     return;
26 }

```

We target the CW308_STM32F3 platform and compile the RM encoder with `arm-none-eabi-gcc` using `-O3`. Under these settings, the RM encoding routine produces the following Cortex-M4 assembly excerpt (one RS symbol per iteration):

The key observation is that the compiler realizes `BITOMASK(message >> i)` using `SBFX` (Signed Bit-Field Extract). Specifically, `sbfx rX, r3, #i, #1` extracts the i -th bit of the byte in `r3` and sign-extends it to a 32-bit value. Therefore, the output register becomes either `0x00000000` (if the extracted bit is 0) or `0xFFFFFFFF` (if the bit is 1), matching exactly the semantics of `BITOMASK`. The subsequent `AND` operations with constants (`0xaaaaaaaa`, `0xcccccccc`, `0xf0f0f0f0`, `0xff00ff00`, `0xffff0000`) select large, highly structured bit patterns depending on the message bit, and the `EOR` chain xors these patterns into the evolving accumulator.

Listing 3: Cortex-M4 `-O3` assembly excerpt for RM encoding (excerpt; abridged).

```

1  080005c4 <encode>:
2  80005c4: f341 0240 sbfx r2, r1, #1, #1
3  80005c8: f341 13c0 sbfx r3, r1, #7, #1
4  80005cc: f002 32cc and.w r2, r2, #0xcccccccc
5  80005d0: 4053 eors r3, r2
6  80005d2: f341 0200 sbfx r2, r1, #0, #1
7  80005d6: f002 32aa and.w r2, r2, #0xaaaaaaaa
8  80005da: 4053 eors r3, r2
9
10 @ ... (similar sbfx/and/eors sequence for bits #2--#4 omitted) ...
11
12 80005fa: f341 1240 sbfx r2, r1, #5, #1
13 80005fe: 6003 str r3, [r0, #0]
14 8000600: f341 1180 sbfx r1, r1, #6, #1
15 8000604: 4053 eors r3, r2
16 8000606: 4059 eors r1, r3
17 8000608: 404a eors r2, r1
18 800060a: 6043 str r3, [r0, #4]
19 800060c: 60c1 str r1, [r0, #12]
20 800060e: 6082 str r2, [r0, #8]
21 8000610: 4770 bx lr

```

From a leakage-analysis perspective, this instruction sequence is particularly favorable to an attacker: each input bit is expanded into a full 32-bit all-zero or all-one value (SBFX output), which in turn drives a cascade of wide-register logical operations. As a result, the Hamming weight and bit transitions of intermediate registers and memory stores become strongly correlated with individual message bits. Since this pattern repeats for each of the 46 RS symbols (and the systematic RS layout fixes which symbols correspond directly to message bytes), the RM encoder provides a highly structured and repeatable source of message-dependent side-channel leakage, enabling straightforward byte- and bit-level recovery in the following sections.

3.2 Empirical Leakage Assessment of RM Encoding

To quantify the practical leakage of the RM-encoding routine, we collected power traces using a PicoScope 3403D and a ChipWhisperer CW308 UFO board equipped with an STM32F303 target board. A ChipWhisperer-Lite was connected in parallel to provide precise trigger generation and to control the execution flow (reset/arming), ensuring consistent trace alignment across measurements.

All measurements in this section target the RM encoder in the official HQC upstream code base ([Tea26]). We compile the implementation with `-Os` and profile only the `rm_encode` routine in isolation (i.e., excluding the rest of the decapsulation/re-encryption path) to characterize whether the implementation exhibits clear message-dependent leakage. In this subsection, we focus solely on *detecting* and quantifying leakage (e.g., via standard leakage tests); message inference and single-trace recovery are treated separately in the subsequent sections.

After establishing the measurement setup, we performed a first-order leakage test using a two-sample Welch’s t -test with a bit-conditioned fixed-vs-fixed (FvF) partitioning. Concretely, we feed `rm_encode` with a single input byte and, for each target bit position $b \in \{0, \dots, 7\}$, form two trace sets: (i) a set where bit b is fixed to 0 while the remaining seven bits are sampled uniformly at random, and (ii) a set where bit b is fixed to 1 while the remaining bits remain random. This isolates the contribution of the selected bit while averaging out effects from the other input bits.

For each bit position, we collected 200 traces per set (400 traces total), and computed both the mean power difference between the two sets and the corresponding Welch t -statistic at each time sample. We use TVLA (Test Vector Leakage Assessment) in the sense of Gilbert *et al.* [GGJR⁺11] to localize leakage points statistically: time samples where $|t| > 4.5$ are typically considered evidence of statistically significant first-order leakage. We report these results per bit to reveal which parts of the RM-encoding instruction sequence exhibit strong, temporally localized data dependence.

Results and interpretation. Figure 2 shows the resulting t -test traces (overlay across the 16 iterations, together with the mean). We observe strong, repeatable leakage points whose locations match the compiler-generated instruction sequence. In particular, the POIs appear in the same order as the bit-processing order in the RM encoder of Listing 2: the routine effectively handles bits in the sequence $7 \rightarrow 0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$, and the t -test peaks arise accordingly.

Moreover, the mean power difference curve closely aligns with the locations where the Welch t statistic exceeds the detection threshold, providing consistent evidence of message-dependent leakage at the same time offsets.

This behavior indicates that leakage associated with each message bit manifests at *distinct* and largely non-overlapping time offsets. Such temporal separation is advantageous to an attacker, as it enables independent per-bit discrimination with minimal interference from other bits.

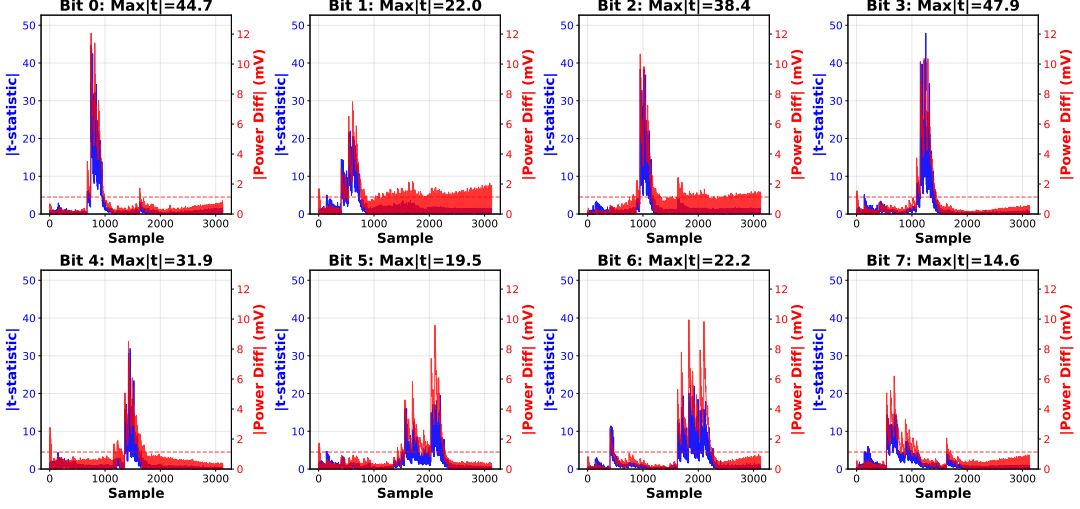


Figure 2: TVLA of RM encoding for one input byte (400 traces/bit: 200 with bit=0 and 200 with bit=1; other bits random). Blue/left: Welch t values; green/right: mean power difference.

Table 1: Per-bit MLP accuracy on RM-encoding traces (320 train / 80 test per bit).

Bit	0	1	2	3	4	5	6	7	Avg.
Accuracy (%)	97.50	98.75	98.75	96.25	98.75	96.25	98.75	97.50	97.81

To confirm that the identified POIs are directly exploitable, we performed a simple profiling experiment using a lightweight MLP classifier per bit. Using 320 traces for training and 80 traces for testing (per bit), we obtain consistently high classification accuracy across all eight bits, as summarized in Table 1.

These results indicate that each message bit can be distinguished reliably from the RM-encoding leakage with only a few hundred profiling traces per bit, confirming that the leakage is strong and readily exploitable.

More importantly, this bit-wise leakage profile directly supports our full-codeword recovery strategy. Since RM encoding is applied independently to each of the 46 RS symbols, accurate and temporally separated bit classification allows the attacker to assemble a noisy estimate of the full 46-byte RS codeword from a single trace. At that point, exact recovery of every byte is no longer necessary: it suffices to remain within the correction radius of the outer RS decoder, which can repair the residual symbol errors and recover the original 128-bit message exactly. In this sense, the strong local leakage observed at the RM level combines with the global error-correction capability of the outer RS layer to create the attack surface exploited in the following sections.

4 Experimental Results

4.1 Experimental Setup

Our experiments use the same measurement platform and acquisition procedure as in the TVLA study of Section 3.2: a ChipWhisperer CW308 UFO board with an STM32F303 target, power traces captured by a PicoScope 3403D at 250 MS/s, and a ChipWhisperer-Lite used for synchronized triggering and execution control (reset/armng) to ensure stable trace alignment.

Table 2: Message recovery performance from RM-encoding leakage using bit-wise CNN classifiers (8 binary CNNs per byte). Results are shown for selected trace budgets. Bit and byte accuracies are arithmetic means over 128 bits and 16 bytes, respectively.

Traces	Bit acc. (%)	Byte acc. (%)	Full msg acc. (%)
100	94.61	78.86	3.80
300	96.01	86.36	13.90
600	97.64	84.72	9.20
800	97.79	88.59	20.30
900	99.61	98.62	90.80
1000	99.60	98.73	92.10
2000	99.50	98.87	93.90
3000	99.51	98.96	95.40
3200	99.52	99.06	96.90
4000	99.52	99.00	96.00

In Section 3.2, we profiled `rm_encode` in isolation on a single input byte to detect and localize leakage (under `-0s`). In contrast, the experiments in this section cover the *full* RM-encoding process as it occurs in the attack pipeline, i.e., RM encoding is executed over all RS symbols rather than a single byte. Concretely, the target runs the HQC-128 reference implementation compiled with `arm-none-eabi-gcc v.10.3.1` at `-O3`, and we collect 5,000 profiling traces with uniformly random 128-bit messages.

4.2 RM-Encoding Message-Recovery Experiment and Results

Dataset, model, and evaluation. We collected a total of 5,000 traces, consisting of 4,000 profiling traces for training and 1,000 traces reserved for evaluation. The input messages are uniformly random 128-bit values, and we train a single RM-byte estimator (internally consisting of eight bit-wise classifiers). Each bit classifier is implemented as a 1D CNN, and we improve robustness using a 5-model ensemble whose outputs are combined (e.g., by majority vote or by averaging posterior probabilities). We evaluate performance as a function of the number of traces used per decision, and report three metrics: (i) bit accuracy (averaged over all bits), (ii) byte accuracy (all 8 bits correct for a byte), and (iii) full-message accuracy (all 16 bytes correct for a 128-bit message).

Results. Table 2 summarizes the message-recovery performance. With 3,200 traces, we reach 96.9% success in recovering the full 128-bit message. Past this point, the full-message rate remains broadly similar with small fluctuations, which is consistent with diminishing returns (and potentially mild overfitting) once the per-bit and per-byte accuracies are already saturated. Note that the full-message metric is stringent: it counts a success only when *all* 128 bits (i.e., all 16 bytes) are recovered correctly. Hence, even tiny residual bit/byte error rates can limit further improvements when aggregated over the entire message.

Tables and comparison summary. Table 3 places our attack in context by comparing it with the SASCA-based single-trace attack of Goy *et al.* [GMGL24]. Both attacks operate in a single-trace setting, but they target different implementation leakage sources and use different inference mechanisms. The SASCA attack models Hamming-weight leakage from RS encoder/decoder operations (notably `gf_mul`) and performs probabilistic inference on a factor graph via belief propagation, requiring 300,000 profiling traces for template construction. In contrast, our approach targets the RM encoder leakage induced by

Table 3: Comparison with the SASCA-based single-trace attack on HQC [GMGL24].

	SASCA [GMGL24]	This work
Target operation	RS encoder/decoder (<code>gf_mul</code>)	RM encoder (BITOMASK)
Leakage model	Hamming weight	Bit-level (per-bit separation)
Analysis method	Factor graph + belief propagation	8 binary CNN classifiers
Implementation	HQC reference (April 2023)	HQC reference (GitLab latest)
Compiler / Optimization	-O3	<code>arm-none-eabi-gcc -O3</code>
Target platform	STM32F407 (Cortex-M4)	STM32F303 (Cortex-M4)
Training(profiled)	300,000 (of 500,000)	3,200 (of 5,000)
Trace reduction	—	≈99%
Success rate	Decoder: 100%; Encoder: 65–81%	Full 128-bit message: 96.9%

BITOMASK-style bit-to-word expansion, leverages per-bit temporal separation, and uses eight binary CNN classifiers for message recovery. Under our experimental setup, this yields comparable single-trace capability with a substantially smaller profiling budget: 3,200 traces (a reduction of roughly 99%) while recovering the full 128-bit message with 96.9% success.

4.3 Impact of RS Post-Correction on F415

Before presenting the STM32F415 results, it is useful to clarify why raw full-message recovery can substantially underestimate the practical attack strength. In a direct message-recovery setting, success requires that all 16 systematic message bytes be recovered correctly at once. Under a simplified model in which each recovered byte is correct independently with probability p , the corresponding success probability is

$$P_{\text{raw}}(p) = p^{16}.$$

By contrast, in the RS-corrected setting the attacker does not need every byte to be correct. For HQC-128, the attacker recovers a noisy estimate of the full 46-byte RS codeword and succeeds whenever the number of erroneous RS symbols does not exceed the correction radius $\delta = 15$. Equivalently, at least $46 - 15 = 31$ RS symbols must be recovered correctly. Under the same idealized independent-byte model, the success probability becomes

$$P_{\text{RS}}(p) = \sum_{k=31}^{46} \binom{46}{k} p^k (1-p)^{46-k}.$$

Figure 3 illustrates the gap between these two success criteria. Although this model does not capture the full complexity of the real attack—for example, byte accuracies are not perfectly identical or independent across positions—it provides useful intuition: raw full-message recovery is an extremely brittle all-or-nothing metric, whereas RS post-correction replaces it with a much softer threshold on the number of residual symbol errors. This is exactly the effect that becomes visible in the STM32F415 experiments below.

To evaluate how much the outer RS decoder amplifies the practical impact of RM-encoding leakage, we additionally test the attack on the noisier STM32F415 platform. We first consider the *raw* message-recovery setting, where the attack succeeds only if all 16 message bytes are recovered correctly without post-processing. In this baseline setting,

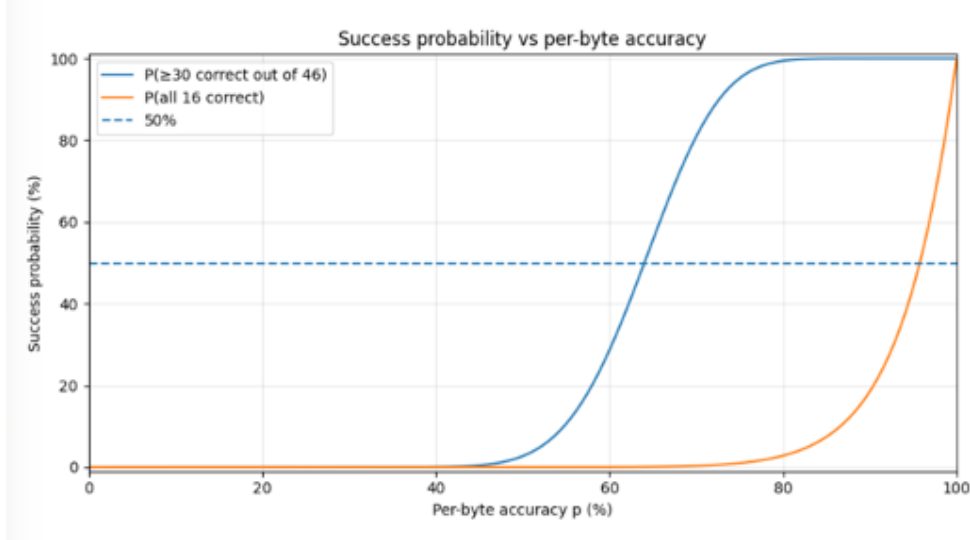


Figure 3: Idealized success probability as a function of per-byte accuracy p . The orange curve corresponds to direct message recovery and requires all 16 systematic message bytes to be correct, i.e., $P_{\text{raw}}(p) = p^{16}$. The blue curve corresponds to RS-corrected recovery for HQC-128 and models the probability that at least 31 out of 46 RS symbols are recovered correctly. Although simplified, this comparison illustrates why RS post-correction can dramatically amplify practical attack success even when raw full-message recovery remains low.

the attack remains fragile: even with 2,000 profiling traces, the best model reaches only 17.2% full-message success on 1,000 evaluation traces.

To evaluate how much the outer RS decoder amplifies the practical impact of RM-encoding leakage, we additionally test the attack on the noisier STM32F415 platform. We first consider the *raw* message-recovery setting, where the attack succeeds only if all 16 message bytes are recovered correctly without post-processing. In this baseline setting, the attack remains fragile: even with 2,000 profiling traces, the best model reaches only 17.2% full-message success on 1,000 evaluation traces.

We then apply the attack in an RS-aware setting. Instead of requiring exact recovery of all 16 systematic message bytes, we recover a noisy estimate of the full 46-byte RS codeword and pass it to the RS decoder. Table 4 shows that this changes the attack regime completely. Even though the raw full-message success remains extremely low for small profiling budgets, the number of symbol errors in the recovered 46-byte word quickly falls below the RS correction radius $\delta = 15$. As a result, RS decoding succeeds with very high probability: already at 60 profiling traces, the actual HQC RS decoder recovers the correct message in 99.5% of the evaluation set, and from 80 traces onward the RS-corrected recovery rate reaches 100%.

This gap between raw and RS-corrected performance is striking. For example, at 60 traces the raw full-message success is still 0%, yet the RS decoder recovers the original message in 99.5% of cases. Even at 500 traces, the raw full-message success is only 12.7%, while the RS-corrected success is already 100%. These results confirm that, on a noisy platform, the outer RS decoder is not merely a reliability component for honest decryption, but a powerful post-correction mechanism for the attacker as well.

Table 4: Effect of RS post-correction on STM32F415. “Full Msg” denotes raw full-message recovery without RS post-processing. “RS dec.” denotes the final message-recovery success after applying the actual HQC RS decoder to the recovered 46-byte RS-codeword estimate.

Traces	Bit Acc. (%)	Byte Acc. (%)	Full Msg (%)	Err. mean	Err. max	RS dec. (%)
20	90.86	45.61	0.00	25.02	39	0.4
25	94.20	61.98	0.00	17.49	29	27.8
30	95.23	67.90	0.00	14.77	26	59.8
35	96.05	72.74	0.00	12.54	21	83.4
40	96.56	75.86	0.00	11.10	22	92.8
60	97.52	82.23	0.00	8.17	18	99.5
80	97.96	85.18	0.00	6.82	15	100.0
100	98.33	87.66	0.20	5.68	14	100.0
200	98.90	91.78	2.10	3.78	11	100.0
300	99.17	93.82	4.70	2.84	10	100.0
400	99.35	95.07	9.40	2.27	9	100.0
500	99.42	95.66	12.70	2.00	7	100.0

4.4 Robustness Under Additional Noise and Jitter

To further quantify the role of RS post-correction, we inject additional noise and jitter into the STM32F415 traces and evaluate how the recovery degrades as the trace quality decreases. Table 5 reports the results. As expected, the raw full-message metric is extremely brittle and remains essentially zero throughout the tested range. However, the RS-corrected metric degrades much more gracefully.

In particular, RS decoding still succeeds in 99.7% of cases at $\sigma = 0.24$, 97.1% at $\sigma = 0.26$, and 88.0% at $\sigma = 0.28$. Only around $\sigma \approx 0.32$, where the mean number of symbol errors exceeds the RS correction capacity of 15, does the RS-decoded success rate collapse sharply. This behavior is fully consistent with the attack model: once the recovered 46-byte RS word stays within the RS correction radius, message recovery remains reliable even though exact raw recovery of all systematic bytes is almost never observed.

These results highlight the main security implication of RS post-correction. The raw byte-wise or message-wise metrics substantially underestimate the real attack capability, because they ignore the error tolerance provided by the outer RS code. From the attacker’s perspective, the relevant threshold is not whether all bytes are predicted correctly, but whether the residual symbol error count stays below $\delta = 15$.

5 Conclusion

HQC has now been selected for standardization, making implementation security on embedded targets a first-order requirement. In this work, we showed that the RM(1, 7) encoding routine constitutes a *systematic* and *cross-implementation* leakage point: the optimized encoder’s bit-to-word expansion (e.g., BITOMASK or compiler-emitted `sbfx`) yields strong, structured, and temporally separable message-dependent leakage. Exploiting this structure, we demonstrated that recovering the full 128-bit message from a *single* decapsulation trace becomes practical with a small profiling budget, directly endangering session-key confidentiality under FO-style decapsulation.

Beyond the immediate message-recovery setting, our results also suggest that this leakage source may have broader implications for other side-channel attack paths on HQC. In particular, once the decrypted message can be recovered reliably, the cost of instantiating higher-level attack primitives may be reduced. We leave such extensions to future work

Table 5: STM32F415 results under additional noise and jitter. “Msg(raw)” is the direct full-message recovery rate without RS post-correction, whereas “RS decode” is the final recovery rate after RS decoding.

σ	Bit (%)	Byte (%)	Msg(raw) (%)	RS decode (%)	Errors mean	Errors max
0.20	98.16	86.58	0.10	100.00	6.17	14
0.22	97.82	84.27	0.00	99.90	7.24	17
0.24	97.40	81.53	0.00	99.70	8.50	17
0.26	96.82	77.81	0.00	97.10	10.21	21
0.28	96.11	73.48	0.00	88.00	12.20	23
0.30	95.20	68.27	0.00	61.10	14.60	26
0.32	94.07	62.11	0.00	28.00	17.43	27
0.34	92.70	55.53	0.00	6.70	20.46	31
0.36	91.09	48.75	0.00	0.60	23.57	33
0.38	89.15	41.64	0.00	0.00	26.84	36
0.40	86.95	34.75	0.00	0.00	30.01	39

and focus here on the core observation we want to document: RM-encoding leakage, when combined with RS post-correction and FO-style re-encryption, yields a particularly strong and practical attack surface in HQC.

A Classifier Configuration (MLP & CNN)

A.1 MLP for Bit-wise Leakage Classification

We train an independent binary MLP per bit on POIs extracted from RM-encoding traces. We use an 80/20 split (320/80 traces per bit) with early stopping.

Table 6: Per-bit MLP hyperparameters (Table 1).

Parameter	Value
hidden_layer_sizes	(64, 32)
activation	ReLU
max_iter	500
early_stopping	True (validation_fraction=0.15)
random_state	42

A.2 CNN Ensemble for Bit-wise Message Recovery

We use eight binary 1D-CNNs per byte and an ensemble of $M = 5$ models. Inputs are standardized (fit on training) and aggregated by averaging softmax posteriors.

Table 7: CNN architectures (summary) and training hyperparameters.

Component	Setting
Architectures	CNN1D: Conv(32,k=5)→MP2 → Conv(64,k=5)→MP2 → Conv(128,k=3)→AAP1 → FC(128)→2 CNN1D _{Small} : Conv(16,k=7)→MP2 → Conv(32,k=5)→AAP1 → FC(64)→2
Loss / Optimizer	CrossEntropy (2-class logits) / AdamW
Learning rate / Weight decay	1×10^{-3} / 1×10^{-4}
Training budget	Max 30 epochs, early stopping patience 5 (val. acc.)
Batching	Train batch 256, inference batch 512
Ensemble / Split	$M = 5$ models; 20% stratified validation (<code>random_state=42</code>)

References

- [AAC⁺22] Gorjan Alagic, Daniel Apon, David Cooper, Quynh Dang, Thinh Dang, John Kelsey, Jacob Lichtinger, Yi-Kai Liu, Carl Miller, et al. Status report on the third round of the nist post-quantum cryptography standardization process. 2022.
- [ABB⁺22] Nicolas Aragon, Paulo Barreto, Slim Bettaieb, Loic Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Santosh Ghosh, Shay Gueron, Tim Güneysu, et al. Bike: bit flipping key encapsulation. 2022.
- [ABC⁺25] Gorjan Alagic, Maxime Bros, Pierre Ciadoux, David Cooper, Quynh Dang, Thinh Dang, John Kelsey, Jacob Lichtinger, Yi-Kai Liu, Carl Miller, et al. Status report on the fourth round of the nist post-quantum cryptography standardization process, 2025.
- [BCL⁺17] Daniel J Bernstein, Tung Chou, Tanja Lange, Ingo von Maurich, Rafael Misoczki, Ruben Niederhagen, Edoardo Persichetti, Christiane Peters, Peter Schwabe, Nicolas Sendrier, et al. Classic mceliece: conservative code-based cryptography. *NIST submissions*, 1(1):1–25, 2017.
- [BDK⁺18] Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-kyber: a cca-secure module-lattice-based kem. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 353–367. IEEE, 2018.
- [Ber25] Pierre-Augustin Berthet. Indepth analysis of a side-channel message recovery attack against frodokem. In *2025 IEEE International Conference on Cyber Security and Resilience (CSR)*, pages 666–671. IEEE, 2025.
- [BHK⁺19] Daniel J Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe. The sphincs+ signature framework. In *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*, pages 2129–2146, 2019.
- [BL17] Daniel J Bernstein and Tanja Lange. Post-quantum cryptography. *Nature*, 549(7671):188–194, 2017.
- [CCJ⁺16] Lily Chen, Lily Chen, Stephen Jordan, Yi-Kai Liu, Dustin Moody, Rene Peralta, Ray A Perlner, and Daniel Smith-Tone. *Report on post-quantum cryptography*, volume 12. US Department of Commerce, National Institute of Standards and Technology . . . , 2016.

- [CDCG22] Brice Colombier, Vlad-Florin Drăgoi, Pierre-Louis Cayrel, and Vincent Grosso. Profiled side-channel attack on cryptosystems based on the binary syndrome decoding problem. *IEEE Transactions on Information Forensics and Security*, 17:3407–3420, 2022.
- [DG25a] Haiyue Dong and Qian Guo. Multi-value plaintext-checking and full-decryption oracle-based attacks on hqc from offline templates. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(4):254–289, 2025.
- [DG25b] Haiyue Dong and Qian Guo. Ot-pca: New key-recovery plaintext-checking oracle based side-channel attacks on hqc with offline templates. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(1):251–274, 2025.
- [DKL⁺18] Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium: A lattice-based digital signature scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 238–268, 2018.
- [FHK⁺18] Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, Zhenfei Zhang, et al. Falcon: Fast-fourier lattice-based compact signatures over ntru. *Submission to the NIST’s post-quantum cryptography standardization process*, 36(5):1–75, 2018.
- [FO99] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In *Annual international cryptology conference*, pages 537–554. Springer, 1999.
- [GAMA⁺25] Philippe Gaborit, Carlos Aguilar-Melchor, Nicolas Aragon, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Edoardo Persichetti, Gilles Zémor, Jurjen Bos, et al. Hamming quasi-cyclic (hqc). *NIST Post-Quantum Standardization, 4th Round; National Institute of Standards and Technology: Gaithersburg, MD, USA*, 2025.
- [GGJR⁺11] Benjamin Jun Gilbert Goodwill, Josh Jaffe, Pankaj Rohatgi, et al. A testing methodology for side-channel resistance validation. In *NIST non-invasive attack testing workshop*, volume 7, pages 115–136, 2011.
- [GLG22] Guillaume Goy, Antoine Loiseau, and Philippe Gaborit. estimating the strength of horizontal correlation attacks in the hamming weight leakage model: a side-channel analysis on hqc kem. In *WCC 2022: The Twelfth International Workshop on Coding and Cryptography*, page WCC_2022_paper_48, 2022.
- [GMGL24] Guillaume Goy, Julien Maillard, Philippe Gaborit, and Antoine Loiseau. Single trace hqc shared key recovery with sasca. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024(2):64–87, 2024.
- [HSC⁺23] Senyang Huang, Rui Qi Sim, Chitchanok Chuengsatiansup, Qian Guo, and Thomas Johansson. Cache-timing attack against hqc. *Cryptology ePrint Archive*, 2023.
- [LNPS20] Norman Lahr, Ruben Niederhagen, Richard Petri, and Simona Samardjiska. Side channel information set decoding using iterative chunking: Plaintext recovery from the “classic mceliece” hardware reference implementation. In

- International Conference on the Theory and Application of Cryptology and Information Security*, pages 881–910. Springer, 2020.
- [MAB⁺18] Carlos Aguilar Melchor, Nicolas Aragon, Slim Bettaieb, Loic Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Edoardo Persichetti, Gilles Zémor, and IC Bourges. Hamming quasi-cyclic (hqc). *NIST PQC Round*, 2(4):13, 2018.
- [Mil86] Victor S Miller. Use of elliptic curves in cryptography. *Advances in Cryptology—CRYPTO’85 Proceedings*, pages 417–426, 1986.
- [MS77] Florence Jessie MacWilliams and Neil James Alexander Sloane. *The theory of error-correcting codes*, volume 16. Elsevier, 1977.
- [NDGJ21] Kalle Ngo, Elena Dubrova, Qian Guo, and Thomas Johansson. A side-channel attack on a masked ind-cca secure saber kem implementation. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 676–707, 2021.
- [RBRC21] Prasanna Ravi, Shivam Bhasin, Sujoy Sinha Roy, and Anupam Chattopadhyay. On exploiting message leakage in (few) nist pqc candidates for practical message recovery attacks. *IEEE Transactions on Information Forensics and Security*, 17:684–699, 2021.
- [RSA78] Ronald L Rivest, Adi Shamir, and Leonard Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978.
- [SGG24] Robin Leander Schröder, Stefan Gast, and Qian Guo. Divide and surrender: Exploiting variable division instruction timing in {HQC} key recovery attacks. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 6669–6686, 2024.
- [Sho94] Peter W Shor. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th annual symposium on foundations of computer science*, pages 124–134. Ieee, 1994.
- [SKL⁺20] Bo-Yeon Sim, Jihoon Kwon, Joohee Lee, Il-Ju Kim, Tae-Ho Lee, Jaeseung Han, Hyojin Yoon, Jihoon Cho, and Dong-Guk Han. Single-trace attacks on message encoding in lattice-based kems. *IEEE Access*, 8:183175–183191, 2020.
- [Tea26] HQC Team. HQC: Official reference implementation (gitlab), commit d622142a50f3ce6b6e1f5b15a5119d96c67194e0. <https://gitlab.com/pqc-hqc/hqc/-/commit/d622142a50f3ce6b6e1f5b15a5119d96c67194e0>, 2026. Accessed: 2026-01-14.
- [VCGS14] Nicolas Veyrat-Charvillon, Benoît Gérard, and François-Xavier Standaert. Soft analytical side-channel attacks. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 282–296. Springer, 2014.